

ResumeAI

Technical Documentation

Full-Stack AI Resume Screening Platform

Stack: FastAPI • Next.js 16 • Supabase • Gemini 2.0 • spaCy • ChromaDB

Version 1.0 • June 15, 2026

Overview

ResumeAI is a production-grade, full-stack resume screening platform that combines classical Natural Language Processing (spaCy, NLTK) with generative AI (Google Gemini 2.0) to rank multiple resumes against a job description. Every analysis is persisted in Supabase PostgreSQL and binary resume files are stored in Supabase Storage, providing a complete candidate history with cross-session search, filtering, and PDF preview.

Contents

1	Architecture Overview	4
1.1	High-Level Component Diagram	4
1.2	Data Flow — Analysis Pipeline	4
2	Technology Stack	5
2.1	Backend	5
2.2	Frontend	6
2.3	Infrastructure & Persistence	7
3	Feature Reference	8
3.1	Resume Screening Pipeline	8
3.1.1	File Upload	8
3.1.2	Text Extraction	8
3.1.3	AI Toggle	9
3.2	Composite Scoring Algorithm	9
3.2.1	Component 1 — Keyword Overlap	9
3.2.2	Component 2 — Semantic (Vector) Similarity	9
3.2.3	Component 3 — AI Skills Match Score	9
3.2.4	Composite Formula	9

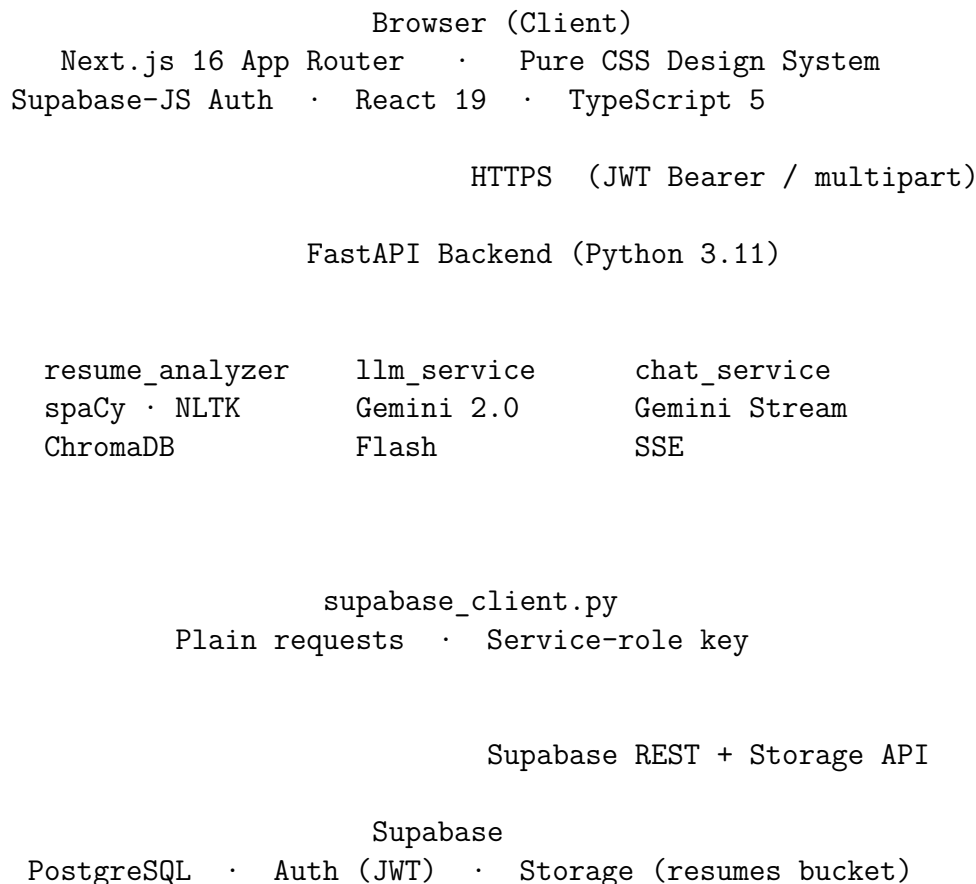
3.2.5	Verdict Thresholds	10
3.3	AI-Powered Features	10
3.3.1	JD Skills Extraction	10
3.3.2	Resume Skills Extraction	10
3.3.3	Skills Comparison	10
3.3.4	Per-Candidate AI Summary	11
3.3.5	AI Ranking Analysis	11
3.3.6	Executive Summary	11
3.4	Name and College Extraction	11
3.4.1	Candidate Name Extraction (<code>extract_candidate_name</code>)	11
3.4.2	College Name Extraction (<code>extract_college_name</code>)	11
3.5	Analysis History	11
3.5.1	Data Model	12
3.5.2	Lazy Loading	12
3.5.3	Session Card UI	12
3.6	Candidate Database	12
3.6.1	Search and Filtering	12
3.6.2	Expandable Row	12
3.6.3	PDF Resume Viewer	13
3.7	AI Chatbot Assistant	13
3.7.1	Architecture	13
3.7.2	Context Injection	13
3.7.3	Markdown Rendering	13
3.8	Authentication System	14
3.8.1	Supported Methods	14
3.8.2	Session Management	14
3.8.3	Server-Side Validation	14
3.8.4	Avatar Component	14
3.9	Excel / CSV Export	15
3.9.1	Server-Side Excel Export (<code>POST /export</code>)	15
3.9.2	Client-Side CSV Export (History Page)	15
4	Backend — Module Reference	15
4.1	<code>main.py</code>	15
4.2	<code>llm_service.py</code>	15
4.3	<code>resume_analyzer.py</code>	15
4.4	<code>supabase_client.py</code>	16
4.5	<code>chat_service.py</code>	16
5	REST API Reference	17
5.1	<code>POST /analyze</code> — Request / Response	17
6	Database Schema	18
6.1	<code>analysis_sessions</code>	18
6.2	<code>candidates</code>	18
6.3	Row Level Security	19
7	Frontend Architecture	19
7.1	App Router Structure	19

7.2	Design System	20
7.3	Authentication Context	20
8	Environment Variables	20
8.1	Backend — <code>backend/.env</code>	20
8.2	Frontend — <code>frontend/.env.local</code>	20
9	Deployment	21
9.1	Backend on Render	21
9.2	Frontend on Vercel	21
9.3	Post-Deployment Configuration	21
10	Cost Summary	22
11	Troubleshooting Reference	22
12	MCP (Model Context Protocol) Integration Guide	23
12.1	How MCP Applies to ResumeAI	23
12.2	MCP Architecture for ResumeAI	23
12.3	Building a Custom ResumeAI MCP Server	24
12.3.1	Installation	24
12.4	Integrating Third-Party MCP Servers	24
12.4.1	Google Drive — Import JDs Directly	24
12.4.2	Slack — Notify on Strong Matches	25
12.4.3	Gmail / Outlook — Receive Resumes	25
12.4.4	Notion / Airtable — Sync Candidate Records	25
12.5	Wiring MCP into the FastAPI Chat Endpoint	25
12.6	Running the MCP Server	27
12.6.1	Development (stdio)	27
12.6.2	Production (HTTP+SSE)	27
12.7	MCP Security Considerations	27
12.8	Recommended MCP Server Registry	28
	Appendix — Scoring Formula Reference	28

1. Architecture Overview

ResumeAI is a decoupled, three-tier application. The browser communicates exclusively with the FastAPI backend over HTTPS using multipart form data for file uploads and JSON for structured responses. The backend is stateless between requests; all persistence is delegated to Supabase.

1.1. High-Level Component Diagram



1.2. Data Flow — Analysis Pipeline

When a user submits a job description and one or more resumes, the following sequence executes:

1. **File Upload:** The frontend POSTs `multipart/form-data` to `/analyze`. The request optionally includes an `Authorization: Bearer <jwt>` header if the user is authenticated.
2. **Text Extraction:** `resume_analyzer.extract_text_from_bytes()` routes each uploaded file to either `PyPDF2` (for PDF) or `docx2txt` (for DOCX).
3. **JD Skills Extraction (AI):** If AI is enabled, the raw JD text is sent to Gemini 2.0 Flash via `llm_service.extract_jd_skills()` which returns structured technical skills, soft skills, tools, and experience level.

4. **Per-Resume Pipeline:** For each resume, in parallel logic:
 - spaCy vector comparison (semantic similarity)
 - NLTK keyword overlap scoring
 - Gemini AI skills comparison (matching / missing skills)
 - Gemini AI per-candidate summary
 - spaCy NER-based name and college extraction
5. **Composite Scoring:** Each resume receives a 0–100 composite score based on a weighted formula (see Section 3.2).
6. **Persistence:** If authenticated, the session metadata is saved to `analysis_sessions`, all candidate results to `candidates`, and each raw resume file to Supabase Storage under `resumes/user_id/session_id/filename`.
7. **AI Ranking & Executive Summary:** Gemini generates a cross-candidate ranking narrative and a 4–6 paragraph executive summary.
8. **Response:** The complete ranked result set is returned as JSON.

2. Technology Stack

2.1. Backend

Technology	Version / Package	Role
Python	3.11.12	Runtime. Pinned in <code>.python-version</code> for dependency compatibility (spaCy, NumPy, ChromaDB).
FastAPI	0.115.0	ASGI web framework. Provides REST API, file upload handling, dependency injection, and Pydantic model validation.
Uvicorn	0.30.6	ASGI server. Runs with <code>-reload</code> in development; <code>-host 0.0.0.0</code> <code>-port \$PORT</code> in production.
spaCy	3.7.6	Industrial-strength NLP. Uses the <code>en_core_web_md</code> model for word vectors, named entity recognition (PERSON, ORG), and part-of-speech tagging.
NLTK	3.9.1	Tokenisation, stopword removal, POS-based keyword frequency counting. Data downloaded on first run into a local <code>nltk/</code> directory.

Technology	Version / Package	Role
ChromaDB	0.5.15	Embedded vector database. Stores document embeddings for semantic search. Configurable between persistent (local) and ephemeral (cloud) modes via the <code>CHROMADB_EPHEMERAL</code> env var.
Google GenAI SDK	$\geq 1.0.0$	Official Python SDK for Google Gemini. Used in both <code>llm_service.py</code> (synchronous structured calls) and <code>chat_service.py</code> (streaming via <code>generate_content_stream</code>).
Gemini 2.0 Flash	<code>gemini-2.0-flash</code>	Default AI model. Configurable via <code>GEMINI_MODEL</code> env var. Free tier: 15 RPM, 1M tokens/day.
PyPDF2	3.0.1	PDF text extraction via page-by-page <code>extract_text()</code> .
docx2txt	0.8	DOCX text extraction.
pandas	2.2.3	Used in the <code>/export</code> endpoint to produce structured <code>.xlsx</code> files.
xlsxwriter	3.2.0	Excel engine for pandas ExcelWriter. Applies column widths.
requests	2.32.3	HTTP client for all Supabase REST API calls. SSL verification is disabled (<code>verify=False</code>) for Windows compatibility; <code>urllib3</code> warnings are suppressed.
python-dotenv	$\geq 1.0.0$	Loads <code>.env</code> file into <code>os.environ</code> at startup.
NumPy	1.26.4	Numerical operations for TF-weighted vector construction and cosine similarity computation.

2.2. Frontend

Technology	Version / Package	Role
Next.js	16 (latest)	React meta-framework. Uses the App Router with React Server Components where applicable. Provides file-based routing, next/font for Google Fonts, and next/image configuration.
React	19	UI component model. Uses hooks (useState , useEffect , useCallback , useMemo , useRef) throughout.
TypeScript	5	Static typing across all frontend files. Strict mode enabled.
@supabase/supabase-js	2.43+	Official Supabase client. Handles: signInWithPassword , signUp , signInWithOAuth (Google), resetPasswordForEmail , real-time auth state via onAuthStateChange .
Pure CSS	—	No CSS framework. Custom design system implemented via CSS custom properties (:root variables) in globals.css . DM Sans, DM Serif Display, and DM Mono Google Fonts.
Google Fonts	DM Sans, DM Serif Display, DM Mono	Typography system. Loaded via Next.js next/font/google for performance and layout-shift prevention.

2.3. Infrastructure & Persistence

Service	Tier	Role
Supabase PostgreSQL	Free (500 MB)	Primary database. Stores analysis_sessions and candidates tables. Row-Level Security (RLS) enforces per-user data isolation.
Supabase Auth	Free	JWT-based authentication. Supports email/password and Google OAuth. JWTs validated server-side via /auth/v1/user .

Service	Tier	Role
Supabase Storage	Free (1 GB)	Binary resume file storage. Private bucket <code>resumes</code> with path schema <code>{user_id}/{session_id}/{filename}</code> .
Render	Free tier	Backend hosting. Python 3.11, auto-deploys from GitHub. Build command installs spaCy model. Spins down after 15 min inactivity.
Vercel	Hobby	Frontend hosting. Auto-deploys on push to <code>main</code> . Reads <code>NEXT_PUBLIC_*</code> environment variables at build time.

3. Feature Reference

3.1. Resume Screening Pipeline

The core feature of ResumeAI is its multi-stage resume screening pipeline accessible at the **Screener** page (/).

3.1.1. File Upload

The `Uploader` component (`frontend/components/Uploader.tsx`) supports both drag-and-drop and click-to-browse upload interactions. It accepts `.pdf` and `.docx` file formats. The job description uploader is configured for single-file selection; the resumes uploader accepts multiple files simultaneously. Files are validated client-side by extension. Server-side, the `content_type` MIME header is used to route to the appropriate text extraction backend.

3.1.2. Text Extraction

`resume_analyzer.extract_text_from_bytes()` detects file type from the MIME `content_type` string:

```

1 def extract_text_from_bytes(file_bytes: bytes, content_type:
  str) -> str:
2     if "pdf" in content_type:
3         reader = PdfReader(io.BytesIO(file_bytes))
4         return "".join(p.extract_text() or "" for p in
           reader.pages)
5     if "wordprocessingml" in content_type or "docx" in
       content_type:
6         return docx2txt.process(io.BytesIO(file_bytes))

```

Listing 1: Text extraction routing logic

PyPDF2 performs page-by-page extraction. DOCX files are processed by `docx2txt` which handles embedded images, tables, and text boxes. Both return plain UTF-8 strings.

3.1.3. AI Toggle

The “Enable AI Analysis (Gemini)” toggle controls whether the Gemini 2.0 Flash API is invoked during the analysis. When disabled, only NLP scoring (spaCy + NLTK) is performed. This reduces latency and API usage when a quick baseline ranking is sufficient. The backend checks `enable_ai` AND `llm_service.is_gemini_configured()` to determine the final `ai_enabled` flag.

3.2. Composite Scoring Algorithm

Each resume receives a single composite score on the interval $[0, 100]$ built from three independently computed components.

3.2.1. Component 1 — Keyword Overlap

spaCy’s `en_core_web_md` model tokenises both the JD and resume text. Tokens are filtered to content words (NOUN, VERB, ADJ, PROPN) that are alphabetic, non-stopword, and have a trained vector. Both token sets are lemmatised and lowercased to produce sets K_{JD} and K_R .

$$\text{keyword_overlap} = \frac{|K_{JD} \cap K_R|}{|K_{JD}|} \quad (1)$$

3.2.2. Component 2 — Semantic (Vector) Similarity

A TF-weighted centroid vector is computed for the JD. For each unique token t appearing c_t times out of N total tokens:

$$\vec{v}_{JD} = \sum_{t \in K_{JD}} \frac{c_t}{N} \cdot \vec{v}_t \quad (2)$$

The resume vector is the unweighted mean of its content token vectors. Cosine similarity between the two vectors yields `vector_similarity`.

$$\text{vector_similarity} = \frac{\vec{v}_{JD} \cdot \vec{v}_R}{\|\vec{v}_{JD}\| \|\vec{v}_R\|} \quad (3)$$

3.2.3. Component 3 — AI Skills Match Score

When AI is enabled, `compare_resume_to_jd()` sends the JD skills summary and resume text to Gemini with a structured prompt requesting a 0–100 integer **Skills Match Score**. The raw integer is normalised to $[0, 1]$ before entering the composite formula.

3.2.4. Composite Formula

$$\text{score} = \begin{cases} 0.6 \times \text{kw} + 0.4 \times \text{vs} & \text{if AI disabled} \\ 0.4 \times \text{kw} + 0.3 \times \text{vs} + 0.3 \times \text{ai} & \text{if AI enabled} \end{cases} \quad (4)$$

where **kw** = keyword overlap, **vs** = vector similarity, **ai** = AI skills match score (all in $[0, 1]$). The result is multiplied by 100 and rounded to one decimal place.

3.2.5. Verdict Thresholds

Score Range	Verdict	CSS Class
$\geq 70\%$	Strong Match	badge-strong
45%–69%	Moderate Match	badge-moderate
$< 45\%$	Weak Match	badge-weak

3.3. AI-Powered Features

All AI features call Google Gemini 2.0 Flash via `llm_service.py` using `google.genai.Client.models.g` with `temperature=0.7`, `max_output_tokens=2048`.

3.3.1. JD Skills Extraction

`extract_jd_skills(jd_text)` prompts Gemini to return a structured list of required technical skills, soft skills, tools, and experience level from the raw JD text. The response is parsed by `_parse_skills_response()` which iterates lines, applies regex cleanup, and populates the `SkillsData` dict.

3.3.2. Resume Skills Extraction

`extract_resume_skills(resume_text)` applies the same structured extraction to a resume. It produces a `SkillsData` dict representing what the candidate claims to possess—independent of the JD.

3.3.3. Skills Comparison

`compare_resume_to_jd(jd_skills, jd_text, resume_text)` generates nine structured output lines:

- **Matching Technical Skills** — JD technical requirements found in resume
- **Missing Technical Skills** — JD technical requirements absent from resume
- **Matching Soft Skills** — JD soft skill requirements found in resume
- **Missing Soft Skills** — JD soft skill requirements absent from resume
- **Matching Tools & Technologies** — JD tool requirements found in resume
- **Missing Tools & Technologies** — JD tool requirements absent from resume
- **Skills Match Score** — Integer 0–100 AI judgment of overall fit
- **Experience Fit** — One-sentence assessment of experience alignment
- **Overall Assessment** — Two-sentence holistic evaluation

`_parse_comparison_response()` handles multi-line values, normalises score to $[0, 1]$, and stores the full raw LLM output in `raw_llm_output` for debugging.

3.3.4. Per-Candidate AI Summary

`generate_resume_summary(resume_text, jd_text)` generates a 3–4 sentence human-readable assessment of key strengths, gaps, and overall fit for each candidate.

3.3.5. AI Ranking Analysis

`generate_ranking_analysis(leaderboard, jd_text)` receives the top 5 candidates with their scores and produces a 2–3 sentence narrative explaining the ranking, why the top candidate stands out, and any concerns.

3.3.6. Executive Summary

`generate_executive_summary(summary_data)` receives aggregate statistics and top-3 candidate info to produce a 4–6 paragraph executive-level summary covering hiring recommendations, skill gap trends, strategic insights, and next steps.

3.4. Name and College Extraction

Implemented in `resume_analyzer.py` using a layered strategy.

3.4.1. Candidate Name Extraction (`extract_candidate_name`)

1. Scan the first 6 non-empty lines for a 2–5 word string that:
 - Contains no digits or contact keywords (@, phone, email, linkedin, etc.)
 - Is title-case, ALL-CAPS, or starts with two capitalised words
2. Fall back to spaCy PERSON NER on the first 500 characters. Accept entities with 2–4 words.

3.4.2. College Name Extraction (`extract_college_name`)

1. Locate the Education section header via case-insensitive regex.
2. Scan the following 20 lines for lines containing education keywords: university, college, institute, iit, nit, bits, iiit, iim, academy, polytechnic, mit, stanford, harvard, oxford, cambridge.
3. If a line contains pipe/comma-separated parts (e.g., *B.Tech / CS / IIT Bombay*), iterate parts in reverse order and return the first that contains an education keyword.
4. Fall back to spaCy ORG NER on the education section, filtered by education keywords.
5. Fall back to regex: `(?:from|at|,)\s+([A-Z][...]{5,60}(?:University|College|...))`

3.5. Analysis History

The **History** page (`/history`) is a per-user persistent log of every analysis run, accessible only when authenticated.

3.5.1. Data Model

Each analysis run creates one `analysis_sessions` row and N `candidates` rows sharing the same `session_id` UUID. The session row captures all aggregate statistics and AI-generated narratives; individual candidate rows carry all per-resume scoring details.

3.5.2. Lazy Loading

The History page fetches only session metadata on load (`GET /sessions`). Candidates within a session are fetched on-demand when a session card is expanded, via `GET /sessions/{id}/candidates`. This reduces initial load time for users with many sessions.

3.5.3. Session Card UI

Each session card displays:

- A radial SVG progress ring showing the average score
- JD filename, creation date, and AI-extracted skill tags
- A stacked horizontal progress bar (green/amber/red) showing the strong/moderate/weak candidate split
- A CSV export button (client-side, no server round-trip)
- Delete button (calls `DELETE /sessions/{id}` which cascades to candidates)

3.6. Candidate Database

The **Candidates** page (`/candidates`) presents all candidates across all sessions in a sortable, filterable flat table.

3.6.1. Search and Filtering

- **Full-text search:** Filters by candidate name, college name, or original filename (case-insensitive, client-side).
- **Verdict filter pills:** All / Strong Match / Moderate Match / Weak Match.
- **Sortable columns:** Score (`score_pct`) and Date (`created_at`), toggling ascending/descending with directional arrows.

All filtering and sorting is performed client-side with `useMemo` for $O(n)$ performance.

3.6.2. Expandable Row

Clicking a table row expands an inline detail panel showing the AI summary, overall assessment, experience fit, skill match pills (matching/missing technical, tools), candidate extracted skills, and a **View Resume** banner.

3.6.3. PDF Resume Viewer

When a resume was stored during analysis, the `button` opens a `PdfViewerModal` rendering an `<iframe>` pointing to `GET /candidates/{id}/resume?token={jwt}`.

The backend proxy pattern is necessary because:

- `<iframe>` elements cannot send `Authorization` headers, so the JWT is passed as a URL query parameter.
- Supabase Storage sets `X-Frame-Options: DENY` on direct object URLs, blocking embedding. The backend fetches file bytes server-side and re-serves them with `X-Frame-Options: SAMEORIGIN` and a permissive `Content-Security-Policy`.

3.7. AI Chatbot Assistant

A floating chatbot panel is rendered globally via the `Chatbot` component mounted in `app/layout.tsx`.

3.7.1. Architecture

The chat is powered by the `POST /chat` endpoint which uses `chat_service.stream_chat()` — an `async` generator that calls `client.models.generate_content_stream()` and yields text chunks. The FastAPI endpoint wraps this in a `StreamingResponse` with MIME type `text/event-stream` (Server-Sent Events).

Each chunk is newline-escaped before SSE framing:

```
1 async def generate():
2     async for chunk in chat.stream_chat(messages,
3         system_prompt):
4         escaped = chunk.replace("\\", "\\\\").replace("\n",
5             "\\n")
6         yield f"data: {escaped}\n\n"
7     yield "data: [DONE]\n\n"
```

Listing 2: SSE chunk framing in main.py

The frontend reads the stream with the Fetch API's `ReadableStream / TextDecoder` pattern, accumulating chunks into a full response string and updating React state on each chunk for live streaming display.

3.7.2. Context Injection

When the user is authenticated, the backend fetches up to 50 candidates and 10 sessions from Supabase and injects them into the Gemini system prompt via `chat_service.build_system_prompt()`. Each candidate is serialised as a structured text block including name, college, scores, matching/missing skills, AI summary, and assessment. The system prompt instructs the model to cite specific scores and skills, use bullet points, and avoid verbosity.

3.7.3. Markdown Rendering

The `renderMarkdown()` function in `Chatbot.tsx` implements a custom streaming markdown renderer (no library dependency). It handles fenced code blocks, headings h1–h6,

horizontal rules, bullet and numbered lists, inline bold/italic/code, and paragraph grouping with blank-line spacers — all producing native React node trees.

3.8. Authentication System

Authentication is implemented with Supabase Auth via the `@supabase/supabase-js` client, wrapped in a React context provider (`AuthProvider` in `frontend/lib/auth.tsx`).

3.8.1. Supported Methods

- **Email/Password:** `signInWithPassword` / `signUp`
- **Google OAuth:** `signInWithOAuth({provider: "google"})` with offline access and consent prompt. Redirects back to the app origin on completion.
- **Password Reset:** `resetPasswordForEmail()` sends a magic-link email. The `/reset-password` page listens for the `PASSWORD_RECOVERY` auth state change event before rendering the update form.

3.8.2. Session Management

The `AuthProvider` calls `supabase.auth.getSession()` on mount to hydrate the initial session. It subscribes to `onAuthStateChange` to handle OAuth redirects, token refreshes, and sign-outs. Session state (`session`, `user`, `avatarUrl`, `displayName`) is propagated via React context.

3.8.3. Server-Side Validation

The backend uses the Supabase Auth REST API to validate JWTs:

```
1 resp = requests.get(  
2     f"{url}/auth/v1/user",  
3     headers={  
4         "apikey": anon_key,  
5         "Authorization": f"Bearer {token}"  
6     },  
7     timeout=10  
8 )  
9 if resp.status_code == 200:  
10     return {"id": data["id"], "email": data.get("email", "")}
```

Listing 3: JWT validation via Supabase Auth REST

The `security = HTTPBearer(auto_error=False)` dependency makes authentication optional on most endpoints — unauthenticated requests proceed but results are not persisted.

3.8.4. Avatar Component

The `Avatar` React component renders a user's profile picture from OAuth (`avatar_url` / `picture` metadata fields) with an `onError` fallback to a CSS-styled letter avatar, ensuring a visible avatar at all times.

3.9. Excel / CSV Export

3.9.1. Server-Side Excel Export (POST /export)

Re-runs the full analysis pipeline and returns a formatted `.xlsx` file via `StreamingResponse`. Uses pandas + `xlsxwriter` with custom column widths: name/college (28 chars), score columns (18 chars), skills columns (35 chars).

3.9.2. Client-Side CSV Export (History Page)

The `exportToExcel()` function in `history/page.tsx` builds a CSV in-memory from already-loaded candidate data, creates a Blob URL, and triggers a synthetic anchor click—no server round-trip required. The filename includes the JD name and ISO date.

4. Backend — Module Reference

4.1. `main.py`

The FastAPI application entry point. Defines CORS middleware, dependency injection for auth, and all API route handlers. Also imports and mounts `resume_analyzer`, `llm_service`, `supabase_client`, and `chat_service`.

4.2. `llm_service.py`

Encapsulates all Gemini API interactions. Key functions:

Function	Description
<code>generate_response(prompt)</code>	Core wrapper. Creates a <code>genai.Client</code> , calls <code>models.generate_content()</code> , returns stripped text or error string prefixed with <code>.</code>
<code>extract_jd_skills(jd_text)</code>	Structured JD skill extraction.
<code>extract_resume_skills(resume_text)</code>	Structured resume skill extraction.
<code>compare_resume_to_jd(...)</code>	9-line structured comparison. Stores raw output.
<code>generate_resume_summary(...)</code>	3-4 sentence candidate assessment.
<code>generate_ranking_analysis(...)</code>	Cross-candidate ranking narrative.
<code>generate_executive_summary(...)</code>	4-6 paragraph executive summary.
<code>_parse_skills_response(response)</code>	Parses the structured skills format.
<code>_parse_comparison_response(response)</code>	Parses 9-line comparison output.

4.3. `resume_analyzer.py`

NLP scoring engine and text processing. Key functions:

Function	Description
<code>extract_text_from_bytes(...)</code>	MIME-type routing to PyPDF2 or docx2txt.
<code>extract_candidate_name(text)</code>	Heuristic + NER name extraction.
<code>extract_college_name(text)</code>	Education-section + NER college extraction.

Function	Description
<code>compute_scores(resume, jd, sc)</code>	Computes keyword overlap, vector similarity, and composite score.
<code>analyze_resume(name, text, ...)</code>	Full per-resume pipeline. Calls all sub-functions and returns complete result dict.
<code>store_document(id, content, meta)</code>	Upserts text into ChromaDB collection.
<code>search_documents(query, n)</code>	Vector similarity search over ChromaDB.
<code>get_keyword_counts(text, tags)</code>	NLTK POS-filtered frequency counts.

4.4. supabase_client.py

Pure-requests Supabase REST client. Uses the `service_role` key for all write operations (bypasses RLS). Key functions:

Function	Description
<code>verify_jwt(token)</code>	Validates Bearer token against Supabase Auth.
<code>upload_resume_pdf(...)</code>	Streams raw bytes to Supabase Storage via POST <code>/storage/v1/object/{bucket}/{path}</code> .
<code>fetch_resume_bytes(path)</code>	Downloads bytes via the authenticated storage endpoint <code>(/storage/v1/object/authenticated/...)</code> .
<code>save_session(id, user_id, data)</code>	Inserts into <code>analysis_sessions</code> .
<code>save_candidate(result, user, session, jd)</code>	Maps result dict to DB columns and inserts into <code>candidates</code> .
<code>get_sessions(user_id, limit)</code>	<code>SELECT * WHERE user_id = ?</code> on sessions.
<code>get_session_candidates(s_id, u_id)</code>	Candidates filtered by session + user.
<code>get_candidates(user_id, limit)</code>	All candidates for a user, ordered by date.
<code>delete_session(session_id, user_id)</code>	Cascading delete: candidates then session.
<code>delete_candidate(id, user_id)</code>	Single candidate delete.
<code>get_candidate_by_id(id, user_id)</code>	Returns id and <code>resume_storage_path</code> for PDF proxy.

4.5. chat_service.py

Gemini streaming chat service. The `stream_chat()` async generator converts the OpenAI-style `role: assistant` to Gemini's `role: model`, builds the `contents` list, and calls

`generate_content_stream()` with a `system_instruction`. Each yielded `chunk.text` string is sent directly to FastAPI's `StreamingResponse`.

5. REST API Reference

All endpoints are served by FastAPI at the configured `NEXT_PUBLIC_API_URL` (default: `http://localhost:8000`).

Method	Path	Description	Auth
GET	<code>/health</code>	Gemini + Supabase status check	No
GET	<code>/debug/auth</code>	Verify JWT and Supabase config	No
POST	<code>/jd/skills</code>	Extract skills from JD file only	No
POST	<code>/analyze</code>	Full screening pipeline	Optional
GET	<code>/candidates/{id}/resume</code>	PDF proxy (JWT via query param)	Required
GET	<code>/sessions</code>	List user's analysis sessions	Required
GET	<code>/sessions/{id}/candidates</code>	Candidates in a session	Required
DELETE	<code>/sessions/{id}</code>	Delete session + candidates	Required
GET	<code>/candidates</code>	All candidates for user	Required
DELETE	<code>/candidates/{id}</code>	Delete single candidate	Required
POST	<code>/export</code>	Re-run analysis, download xlsx	Optional
POST	<code>/chat</code>	Streaming AI chat (SSE)	Optional
GET	<code>/search?q=&n=</code>	Semantic search over ChromaDB	No

5.1. POST /analyze — Request / Response

Request: `multipart/form-data`

- `jd_file` — single PDF or DOCX
- `resumes` — one or more PDF or DOCX files
- `enable_ai` — boolean string (`"true"` / `"false"`)

Response: `application/json` — `AnalyzeResponse`

- `results[]` — sorted array of `ResumeResult` objects
- `stats` — aggregate totals (total, avg, strong, moderate, weak)
- `jd_skills` — AI-extracted JD requirements
- `ranking_analysis` — AI narrative (null if AI disabled)
- `executive_summary` — AI summary (null if AI disabled)
- `session_id` — UUID of the created session

- `saved_to_db` — bool; true only if authenticated and Supabase configured

6. Database Schema

6.1. analysis_sessions

Column	Type	Description
<code>id</code>	<code>uuid</code>	Primary key; also used as <code>session_id</code> in candidates
<code>created_at</code>	<code>timestampz</code>	Analysis timestamp
<code>user_id</code>	<code>uuid</code>	FK to <code>auth.users</code> (RLS key)
<code>jd_filename</code>	<code>text</code>	Original JD filename
<code>jd_experience_level</code>	<code>text</code>	AI-extracted required experience
<code>jd_technical_skills</code>	<code>text[]</code>	AI-extracted required technical skills
<code>jd_soft_skills</code>	<code>text[]</code>	AI-extracted required soft skills
<code>jd_tools</code>	<code>text[]</code>	AI-extracted required tools
<code>total_candidates</code>	<code>integer</code>	Number of resumes analysed
<code>avg_score</code>	<code>numeric(5,1)</code>	Mean composite score across candidates
<code>strong_count</code>	<code>integer</code>	Count of candidates with score ≥ 70
<code>moderate_count</code>	<code>integer</code>	Count of candidates with score 45–69
<code>weak_count</code>	<code>integer</code>	Count of candidates with score < 45
<code>ranking_analysis</code>	<code>text</code>	AI ranking narrative
<code>executive_summary</code>	<code>text</code>	AI executive summary
<code>ai_enabled</code>	<code>boolean</code>	Whether AI was active during this run

6.2. candidates

Column	Type	Description
<code>id</code>	<code>uuid</code>	Primary key
<code>session_id</code>	<code>uuid</code>	FK to <code>analysis_sessions.id</code>
<code>user_id</code>	<code>uuid</code>	FK to <code>auth.users</code>
<code>file_name</code>	<code>text</code>	Original resume filename
<code>candidate_name</code>	<code>text</code>	Extracted full name (nullable)
<code>college_name</code>	<code>text</code>	Extracted institution name (nullable)
<code>score_pct</code>	<code>numeric(5,1)</code>	Composite score 0–100
<code>verdict</code>	<code>text</code>	Strong / Moderate / Weak Match
<code>keyword_overlap</code>	<code>numeric(5,1)</code>	Keyword overlap %
<code>vector_similarity</code>	<code>numeric(5,1)</code>	Semantic similarity %
<code>skills_match_score</code>	<code>numeric(5,1)</code>	AI skills score % (null if AI off)
<code>matched_keywords</code>	<code>text[]</code>	JD lemmas found in resume
<code>missing_keywords</code>	<code>text[]</code>	JD lemmas absent from resume (top 20)
<code>matching_technical</code>	<code>text[]</code>	Matching technical skills
<code>missing_technical</code>	<code>text[]</code>	Missing technical skills

Column	Type	Description
matching_soft	text[]	Matching soft skills
missing_soft	text[]	Missing soft skills
matching_tools	text[]	Matching tools/technologies
missing_tools	text[]	Missing tools/technologies
experience_fit	text	AI experience alignment sentence
overall_assessment	text	AI holistic two-sentence evaluation
resume_technical	text[]	Candidate's own technical skills
resume_soft	text[]	Candidate's own soft skills
resume_tools	text[]	Candidate's own tools
experience_level	text	Candidate's stated experience
ai_summary	text	AI 3–4 sentence candidate summary
jd_experience_level	text	Required experience (denormalised)
jd_technical_skills	text[]	Required technical skills (denormalised)
resume_storage_path	text	Supabase Storage path (nullable)

6.3. Row Level Security

Both tables have RLS enabled. Three policies each: SELECT, INSERT, DELETE — all guarded by `auth.uid() = user_id`. The backend's service-role key bypasses RLS for all writes.

7. Frontend Architecture

7.1. App Router Structure

frontend/	
app/	
layout.tsx	Global layout: AuthProvider + Navbar + Chatbot
page.tsx	/ - Screener page
globals.css	Design system: CSS custom properties + all styles
history/page.tsx	/history - Analysis history
candidates/page.tsx	/candidates - Candidate database
reset-password/	
page.tsx	/reset-password - Password reset flow
components/	
Navbar.tsx	Sticky navigation with auth state
AuthModal.tsx	Sign in / sign up / forgot password modal
ResumeCard.tsx	Per-resume collapsible result card
Uploader.tsx	Drag-and-drop file uploader
Chatbot.tsx	Floating AI chat panel
ui.tsx	Shared primitives: VerdictBadge, ScoreBar, Pill, StatCard, Section, Collapsible
lib/	
api.ts	Typed FastAPI HTTP client (all fetch calls)
auth.tsx	Supabase Auth context + useAuth hook

7.2. Design System

The entire UI is styled with pure CSS custom properties defined in `globals.css`. No CSS framework, no Tailwind (the `postcss.config.js` explicitly sets `plugins: {}`). The design uses a dark theme with eight primary variables:

```
1 :root {
2   --bg:           #0f1117;    /* Page background */
3   --bg-raised:    #171b26;    /* Panel background */
4   --bg-card:      #1e2330;    /* Card background */
5   --bg-hover:     #252c3d;    /* Hover state */
6   --border:       #2d3448;    /* Default border */
7   --border-light: #3a4260;    /* Elevated border */
8   --text-primary: #f0f2f8;    /* Primary text */
9   --accent:       #f5a623;    /* Amber accent */
10  --accent-green:  #34d399;    /* Success/match */
11  --accent-amber:  #fbbf24;    /* Warning/moderate */
12  --accent-red:    #f87171;    /* Error/weak/miss */
13 }
```

Listing 4: Core design tokens

Typography: DM Serif Display (headings), DM Sans (body), DM Mono (code, labels, scores).

7.3. Authentication Context

`useAuth()` returns: `supabase`, `session`, `user`, `loading`, `avatarUrl`, `displayName`, and the auth action functions. It is consumed in: `Navbar`, `page.tsx`, `history/page.tsx`, `candidates/page.tsx`, `Chatbot`, `AuthModal`, and `reset-password/page.tsx`.

8. Environment Variables

8.1. Backend — `backend/.env`

Variable	Required	Description
GEMINI_API_KEY	Yes	Google AI Studio API key
GEMINI_MODEL	No	Default: <code>gemini-2.0-flash</code>
SUPABASE_URL	Yes	Supabase project URL
SUPABASE_SERVICE_KEY	Yes	service_role secret key
SUPABASE_ANON_KEY	Yes	anon public key
FRONTEND_URL	Production	Vercel URL for CORS
CHROMADB_EPHEMERAL	No	<code>true</code> for cloud (ephemeral storage)

8.2. Frontend — `frontend/.env.local`

Variable	Required	Description
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	anon public key
NEXT_PUBLIC_API_URL	Yes	FastAPI backend URL

Security Note

Never commit either `.env` or `.env.local` to version control. The `SUPABASE_SERVICE_KEY` grants unrestricted database access and must remain server-side only. Both files are listed in `.gitignore`.

9. Deployment

9.1. Backend on Render

Setting	Value
Runtime	Python 3 (reads <code>backend/.python-version</code> → 3.11.12)
Root Directory	<code>backend</code>
Build Command	<code>pip install -r requirements.txt && python -m spacy download en_core_web_md</code>
Start Command	<code>uvicorn main:app -host 0.0.0.0 -port \$PORT</code>
Instance Type	Free (spins down after 15 min inactivity)

Python Version Pinning

The `backend/.python-version` file pins Python to 3.11.12. This is required because spaCy 3.7.x, NumPy 1.26.x, and ChromaDB 0.5.x do not support Python 3.13+. Render reads this file automatically to select the build-time Python version.

9.2. Frontend on Vercel

Root directory is set to `frontend`. Vercel auto-detects Next.js and sets the framework preset. The `NEXT_PUBLIC_*` variables must be added in the Vercel project settings before the first deployment as they are inlined at build time.

9.3. Post-Deployment Configuration

After both services are live:

1. Set `FRONTEND_URL` in Render to the Vercel URL.
2. In Supabase → Authentication → URL Configuration: set Site URL and add `https://your-app.vercel.app/**` to Redirect URLs.

10. Cost Summary

Service	Plan	Monthly Cost
Google Gemini API	Free tier (15 RPM, 1M tokens/day)	\$0
Supabase	Free tier (500 MB DB, 1 GB storage)	\$0
Render	Free tier (750 h/month, cold starts)	\$0
Vercel	Hobby (100 GB bandwidth)	\$0
Total		\$0/month

11. Troubleshooting Reference

Symptom	Resolution
“Checking backend...” never resolves	Free-tier Render cold start. Wait 30–60 s. Check Render logs for Python/spaCy import errors.
“Gemini API key not configured”	Verify <code>GEMINI_API_KEY</code> in Render env vars. Test the key at https://aistudio.google.com .
<code>saved_to_db: false</code> after analysis	User is not signed in, or JWT validation failed. Check uvicorn terminal for <code>verify_jwt</code> error output.
Build fails with numpy/spaCy errors	Python version is ≥ 3.12 . Confirm <code>backend/.python-version</code> contains exactly <code>3.11.12</code> .
CORS errors in browser console	<code>FRONTEND_URL</code> in Render does not match the Vercel URL exactly (check trailing slash).
Google OAuth not redirecting correctly	Supabase → Authentication → URL Configuration: add <code>https://your-app.vercel.app/**</code> to Redirect URLs.
PDF viewer shows blank / error	<code>resume_storage_path</code> is null — the analysis was run before storage integration or while unauthenticated. Re-run the analysis while signed in.
Chatbot shows “ Gemini API error: 429”	Free-tier rate limit exceeded (15 RPM). Wait 60 s or upgrade billing.
History page empty after analysis	Analysis was run while not signed in. Sign in first, then re-run.

12. MCP (Model Context Protocol) Integration Guide

What is MCP?

The **Model Context Protocol (MCP)** is an open standard that enables AI models to connect to external tools and data sources through a standardised server/client interface. MCP servers expose *tools*, *resources*, and *prompts* over a lightweight JSON-RPC transport (HTTP+SSE or stdio). Clients — such as Claude, IDEs, or custom applications — discover and invoke those tools at inference time, allowing the model to read files, query databases, call APIs, and take actions without hard-coded integrations.

12.1. How MCP Applies to ResumeAI

ResumeAI's chatbot currently draws all its context from a serialised snapshot of the user's candidate database embedded in the Gemini system prompt. Replacing this snapshot approach with live MCP tool calls unlocks several capabilities:

- **Live data:** The model calls a tool at query time instead of reading a potentially stale context snapshot, ensuring answers always reflect the current database state.
- **External integrations:** MCP servers for Google Drive, Slack, Outlook, or Notion can be wired in without modifying application code — the model learns to invoke them from their schema definitions.
- **Actions:** Beyond reading, MCP tools can *write* — creating Slack notifications when a Strong Match is found, adding calendar events for interviews, or writing candidate evaluations back to a CRM.
- **Scalability:** Tool invocations are lazy; the model only fetches data it actually needs, eliminating the 50-candidate context window cap.

12.2. MCP Architecture for ResumeAI

Claude / Gemini (AI Model)

MCP JSON-RPC (stdio or HTTP+SSE)

MCP Host (ResumeAI backend)

resumeai-mcp	third-party MCPs
(custom)	Google Drive
	Slack
tools:	Outlook/Gmail
- search_	Notion
candidates	Asana / Jira
- get_	Salesforce
session	

- compare
- export

Supabase PostgreSQL · Supabase Storage

12.3. Building a Custom ResumeAI MCP Server

The custom MCP server exposes live Supabase data as tools the AI model can call on demand. It is implemented as a standalone Python process using the mcp SDK.

12.3.1. Installation

```
1 pip install mcp
```

Listing 5: Install the MCP Python SDK

12.4. Integrating Third-Party MCP Servers

Third-party MCP servers connect ResumeAI to external platforms without writing custom integration code.

12.4.1. Google Drive — Import JDs Directly

MCP Server	https://drivemcp.googleapis.com/mcp/v1
Auth	Google OAuth 2.0
Use case	Ask the AI: “Use the Q3 SDE Job Description from my Drive” — the model calls <code>drive.search_files</code> to locate the file, <code>drive.read_file</code> to fetch its text, then passes it directly to the screening pipeline.

```

1 # In chat_service.py      model-driven Drive      Analyze flow
2 mcp_servers = [
3     {
4         "type": "url",
5         "url": "https://drivemcp.googleapis.com/mcp/v1",
6         "name": "google-drive",
7     },
8     {
9         "type": "url",
10        "url": "http://localhost:8001/mcp", # your resumeai
11        MCP server
12        "name": "resumeai",
13    }
14 ]
15 # The model can now execute:

```



```

16 # 1. drive.search_files(query="Q3 SDE JD")
17 # 2. drive.read_file(file_id="1abc...")
18 # 3. resumeai.screen_resumes(jd_text=<content>,
    resume_paths=[...])

```

Listing 6: Calling /analyze with a Google Drive JD via MCP

12.4.2. Slack — Notify on Strong Matches

MCP Server	Slack MCP (community)
Auth	Slack Bot OAuth token
Use case	After each analysis, the model automatically posts a summary of Strong Match candidates to a designated #recruiting Slack channel using the slack.post_message tool.

12.4.3. Gmail / Outlook — Receive Resumes

Inbound resume attachments forwarded to a monitored inbox can be fetched by the model using `gmail.list_messages` and `gmail.get_attachment`, bypassing the need for manual file upload entirely.

12.4.4. Notion / Airtable — Sync Candidate Records

After screening, the model can push structured candidate data (name, score, verdict, matching skills) to a Notion database or Airtable base using their respective MCP servers, creating a persistent, shareable hiring tracker outside the ResumeAI UI.

12.5. Wiring MCP into the FastAPI Chat Endpoint

To pass MCP server configurations through to the AI model inside `chat_service.py`, extend the `stream_chat` function signature and include the `mcp_servers` list in the API call configuration:

```

1  async def stream_chat(
2      messages: list[dict],
3      system_prompt: str,
4      mcp_servers: list[dict] | None = None, # <-- new parameter
5  ) -> AsyncGenerator[str, None]:
6      client = genai.Client(api_key=GEMINI_API_KEY)
7
8      # Build request config      add mcp_servers when provided
9      config: dict = {
10         "system_instruction": system_prompt,
11         "temperature": 0.7,
12         "max_output_tokens": 1024,
13     }
14     if mcp_servers:
15         config["tools"] = [{"mcp": {"servers": mcp_servers}}]
16

```

```

17     response = client.models.generate_content_stream(
18         model      = GEMINI_MODEL,
19         contents   = gemini_history,
20         config     = config,
21     )
22     for chunk in response:
23         if chunk.text:
24             yield chunk.text

```

Listing 7: Extended chat_service.py with MCP server support

In `main.py`, construct the MCP server list based on connected services and user authentication state before calling `stream_chat`:

```

1  @app.post("/chat")
2  async def chat_endpoint(body: ChatRequest,
3                          user: dict | None =
4                              Depends(get_current_user)):
5
6      mcp_servers = []
7
8      # Always add the local ResumeAI MCP server
9      mcp_servers.append({
10         "type": "url",
11         "url":  "http://localhost:8001/mcp",
12         "name": "resumeai",
13     })
14
15     # Add Google Drive MCP if the user has connected it
16     if user and user.get("drive_token"):
17         mcp_servers.append({
18             "type": "url",
19             "url":  "https://drivemcp.googleapis.com/mcp/v1",
20             "name": "google-drive",
21         })
22
23     async def generate():
24         async for chunk in chat.stream_chat(
25             messages      = [m.dict() for m in body.messages],
26             system_prompt = chat.build_system_prompt([], []),
27             mcp_servers    = mcp_servers,
28         ):
29             escaped = chunk.replace("\\",
30                                   "\\").replace("\n", "\\n")
31             yield f"data: {escaped}\n\n"
32             yield "data: [DONE]\n\n"
33
34     return StreamingResponse(generate(),
35                             media_type="text/event-stream")

```

Listing 8: Constructing MCP server list in main.py

12.6. Running the MCP Server

12.6.1. Development (stdio)

```
1 # Start the MCP server on stdio      the AI client connects
   directly
2 python resumeai_mcp_server.py
```

Listing 9: Run MCP server in stdio mode (development)

12.6.2. Production (HTTP+SSE)

For production, expose the MCP server as an HTTP endpoint. The **FastMCP** class supports **streamable-http** transport natively:

```
1 # Start on port 8001, separate from the main FastAPI process
2 uvicorn resumeai_mcp_server:mcp.app --host 0.0.0.0 --port 8001
```

Listing 10: Run MCP server over HTTP+SSE (production)

On Render, deploy it as a second Web Service pointing to the same repo with start command: `uvicorn resumeai_mcp_server:mcp.app -host 0.0.0.0 -port $PORT`.

12.7. MCP Security Considerations

Authentication & Authorisation

Every MCP tool that touches the Supabase database must validate the calling user's identity. Never expose raw service-role access through an MCP tool — always scope queries to `user_id`. For third-party MCP servers (Google Drive, Slack), use per-user OAuth tokens stored securely in Supabase or an encrypted secrets store; never hard-code credentials.

Risk	Mitigation
Prompt injection via MCP tool results	Sanitise all strings returned by MCP tools before inserting into model context. Reject tool results containing instruction-like patterns.
Over-permissioned tools	Apply principle of least privilege. The <code>search_candidates</code> tool must only return data for the authenticated <code>user_id</code> .
Tool call exfiltration	Log all MCP tool invocations server-side. Alert on unusual patterns (e.g., bulk data exports, calls to unexpected file paths).
Third-party MCP server trust	Only connect to MCP servers from trusted vendors or self-hosted instances. Verify server identity via TLS and check the MCP registry.
Credential leakage in tool params	Never pass raw API keys or tokens as tool arguments. The server should resolve credentials internally from environment variables.

12.8. Recommended MCP Server Registry

The following publicly available MCP servers are recommended for extending ResumeAI:

Service	MCP Server URL	ResumeAI Use Case
Google Drive	<code>drivemcp</code>	Fetch JD and resume files directly
Gmail	Google Workspace MCP	Receive resumes from inbound emails
Slack	Community Slack MCP	Post Strong Match summaries to <code>#recruiting</code>
Notion	Notion MCP	Sync candidate records to hiring tracker
Google Calendar	Google Workspace MCP	Schedule interviews for shortlisted candidates
Airtable	Airtable MCP	Export ranked results to an Airtable base
Salesforce	Salesforce MCP	Push candidate data to CRM pipeline
Jira	Atlassian MCP	Create hiring tickets for each Strong Match

Appendix — Scoring Formula Reference

Complete Scoring Formula

Without AI:

$$\text{score_pct} = 100 \times \left(0.6 \times \frac{|K_{JD} \cap K_R|}{|K_{JD}|} + 0.4 \times \frac{\vec{v}_{JD} \cdot \vec{v}_R}{\|\vec{v}_{JD}\| \|\vec{v}_R\|} \right)$$

With AI:

$$\text{score_pct} = 100 \times \left(0.4 \times \frac{|K_{JD} \cap K_R|}{|K_{JD}|} + 0.3 \times \frac{\vec{v}_{JD} \cdot \vec{v}_R}{\|\vec{v}_{JD}\| \|\vec{v}_R\|} + 0.3 \times \frac{\text{ai_score}}{100} \right)$$

Where K_{JD} and K_R are lemmatised content-word sets from the JD and resume respectively; $\vec{v}_{JD}$ is the TF-weighted spaCy word vector centroid of the JD; $\vec{v}_R$ is the unweighted mean spaCy word vector of the resume; and `ai_score` is the integer 0–100 returned by Gemini’s structured comparison prompt.